Comparing Reasoning Strategies for Agentic Travel Planning

Akhil Elamanchili

1. Introduction

Everyone loves travelling but it often involves hours or days of planning. You have to search for flights, hotels, attractions, what the weather is like, all within a budget. Travel planning is essentially a multi-step reasoning problem. A request such as “5 days in Paris, budget $2,000, I love museums” requires an agent to coordinate at least four subtasks, flight search, hotel selection, weather forecasting, and attraction discovery, before outputting a coherent, constraint-satisfying itinerary. A single-shot large language model prompt can produce plausible-sounding but hallucinated prices, unavailable hotels, or missing days because it has no access to live data and no mechanism for self-correction.

There has been recent work on LLM agents that have proposed several reasoning strategies to address this. ReAct (Yao et al., 2023) interleaves chain-of-thought reasoning with tool calls in a single loop. Plan-then-Execute (Wang et al., 2023) separates planning from execution into distinct phases. Self-Critique / Reflexion (Shinn et al., 2023) adds an iterative critique-and-refine step. Despite growing interest in these strategies, I wanted to compare these strategies on identical tasks to see which reasoning strategy is best suited for an agentic travel planner.

I implemented all four strategies from scratch using the OpenAI SDK's function-calling API (no agent frameworks), and evaluated them on 40 structured scenarios. The central research question is: **does the reasoning strategy matter for constraint-satisfying travel planning, or is GPT-4o good enough with a zero-shot prompt?**

2. Approach

2.1 Model and Tool Layer

All strategies use the same foundation: GPT-4o as the backbone LLM, four deterministic tools, and a single Pydantic output schema (ItineraryResult). This design ensures that any performance difference is caused by the reasoning loop rather than differences in data access or model capability.

The four tools I implemented are **search_flights**, **search_hotels**, **get_weather**, and **get_attractions**, and each extends a BaseTool abstract base class and exposes a JSON Schema parameters_schema and a run() method. The run method dispatches to mock or live mode; live mode calls SerpAPI Google Flights, Booking.com via RapidAPI, OpenWeatherMap, and Geoapify Places. This dual-mode design enables fully reproducible offline evaluation while preserving the ability to test against real APIs.

2.2 Strategy Implementations

Zero-Shot Baseline. A single GPT-4o call with a structured system prompt and no tool access. It serves as the lower-bound reference point and measures how much grounding in real data actually matters.

ReAct. A loop of up to 15 iterations with tool_choice="required", which forces a function call every step and prevents the LLM from emitting plain-text reasoning that would silently break the loop. Termination is controlled by a synthetic finish_itinerary control-flow tool defined inline, so the model can signal completion without producing a fragile free-form JSON response.

Plan-then-Execute. Three sequential LLM calls: (1) produce a structured {"steps": [...]} plan, (2) execute each step's tool call in sequence, (3) synthesize all tool results into the final itinerary. The plan parser handles both the wrapped object and bare array response formats.

Self-Critique. Five phases: extract query parameters, call all four tools in parallel, draft an itinerary, critique it (score 1–10 plus issues list), and optionally refine. Refinement is skipped when the critique score is ≥ 8 and the issues list is empty, which avoids any unnecessary token spend on already-good drafts.

2.3 Output Schema and Pydantic Coercion

All strategies return the same ItineraryResult Pydantic model. GPT-4o does often produce inconsistent field formats across runs and strategies so I implemented several validators:

- stops returned as 'Non-stop' or 'Direct' strings → coerced to integer 0
- meals returned as dicts with description/cost keys instead of plain strings → extracted to strings
- attractions returned as null instead of [] on some preference scenarios → coerced to empty list
- attraction.category occasionally omitted → defaulted to empty string

2.4 Evaluation Design

40 scenarios span four categories of increasing difficulty:

- **Simple (10):** Single destination, no constraints. Tests structural completeness.
- **Budget (10):** Explicit total budget, per-flight cap, or per-night hotel cap. Tests numeric constraint adherence.
- **Multi-City (10):** 2–3 city trips (e.g., Paris + Rome, Tokyo + Paris). Tests that all cities appear in output.
- **Preference (10):** User interests (museums, food, landmarks). Tests that attraction categories match stated interests.

Two rule-based metrics scores were used for each run:

- **Goal Completion(0–1)** measures the fraction of expected output fields that are non-empty (flights list, hotel name, daily_plan, weather_summary).
- **Constraint Satisfaction(0–1)** measures fraction of constraints met: total cost ≤ budget, all destination city names present in output text, and at least one attraction category matching stated preferences.

3. Results

3.1 Overall Performance

Strategy	Goal Completion	Constraint Satisfaction	Average Latency	Average Tokens	LLM Calls/Run
Zero-Shot Baseline	100%	82.5%	18.0 s	1,614	1
Plan-then-Execute	100%	97.5%	16.1 s	5,086	3
Self-Critique	96.2%	97.5%	37.6 s	12,120	4–5
ReAct	**100%**	**100%**	**16.0 s**	**6,564**	**6–10**

Table 1. Overall evaluation results with mock mode, GPT-4o, seed=42, 40 scenarios per strategy.

3.2 Per Category Constraint Satisfaction

Strategy	Simple	Budget	Multi-City	Preference
Zero-Shot Baseline	100%	90%	100%	40%
Plan-then-Execute	100%	90%	100%	100%
Self-Critique	100%	90%	100%	100%
ReAct	**100%**	**100%**	**100%**	**100%**

Table 2. Constraint satisfaction broken down by scenario category.

3.3 Key Findings

Finding 1. ReAct achieves perfect scores at a reasonable token cost. ReAct is the only strategy to reach 100% on both metrics. Its adaptive loop lets it recover from unexpected tool results. For example, when a hotel search returns no results for an obscure destination, it can re-invoke the tool with relaxed parameters. Plan-then-Execute commits to a fixed plan upfront and cannot recover from such failures.

Finding 2. Plan-then-Execute is the best cost/quality tradeoff. At 3 LLM calls and 5,086 tokens, it achieves 97.5% constraint satisfaction which is just below ReAct, but at roughly half the token cost. The only failures were budget scenarios where the synthesis LLM selected a slightly over-budget option despite being told the cap. For production systems where latency and cost are primary concerns, Plan-then-Execute is the practical choice.

Finding 3. Self-Critique is the most expensive with the weakest marginal return. At 12,120 average

tokens (2× ReAct, 7.5× Baseline) and 37.6 s average latency, it scored identically to Plan-then-Execute on constraint satisfaction and worse on goal completion (96.2%) due to multi-city hotel normalization edge cases. When the underlying tool data is already deterministic and high-quality, the critique-refine loop provides little value.

Finding 4. The zero-shot baseline is surprisingly strong on structured tasks. Despite having no tool access, the baseline achieves 100% goal completion by leveraging GPT-4o's world knowledge about popular destinations. It fails specifically on preference matching (40% constraint satisfaction) because it cannot filter attraction categories without real data. This suggests that for simple itinerary generation without hard constraints, a tool-using agent may be over-engineering the solution.

Finding 5. Strategies diverge most on constrained and preference-heavy tasks. All four strategies perform identically on simple scenarios. The performance gap only appears when queries introduce hard numeric constraints or category-specific preferences, where grounding in real tool data becomes necessary.

3.4 Implementation Challenges

I faced several challenges during development:

- **Tool API Access**: Many travel data providers either require paid subscriptions, have prohibitively low rate limits, or expose only aggregated data unsuitable for per-query tool calls.
- **ReAct loop termination:** Initial use of tool_choice="auto" allowed GPT-4o to emit plain-text reasoning steps ('I will now search for flights...') instead of tool calls, causing 0% goal completion on evaluation. Switching to tool_choice="required" forced a function call every iteration.
- **Self-Critique multi-city normalization:** The LLM inconsistently returns destination as a list instead of a string, daily_plan as a per-city dict instead of a flat list, and weather_summary as a per-city dict for multi-city trips. Added case-specific normalization in _build_result().

4. Ethics and Limitations

1. **Hallucination in live mode.** The mock evaluation environment uses deterministic data, but live mode integrates real APIs with real-time pricing and availability. Even with grounding, GPT-4o may synthesize plausible-but-wrong information when tools return partial results. Users relying on AI-generated itineraries for actual bookings risk booking non-existent hotels, paying incorrect prices, or missing flight connection requirements. Clear disclaimers and human review before any booking are essential.
2. **Bias in attraction recommendations.** The Geoapify Places API and the LLM's world knowledge are biased toward well-documented, English-language tourist destinations. Smaller cities, non-Western destinations, and venues catering to local rather than tourist populations are likely underrepresented. The system may systematically recommend more expensive, tourist-oriented options over authentic local alternatives.
3. **Metric limitations.** Both evaluation metrics are rule-based and cannot detect hallucinated venue names, incorrect dates, or logistically infeasible day plans (e.g., scheduling sites 40 km apart with a 15-minute window). A strategy can score 100% while still producing an itinerary that would fail in

practice. LLM-as-judge evaluation would be required to catch these semantic failures.

4. **Data privacy.** Live mode queries send user travel preferences and date ranges to third-party APIs (SerpAPI, RapidAPI, OpenWeatherMap, Geoapify). If used at scale, users should be informed of this data sharing, and query logs should not be retained beyond the session.

5. Future Work

- **LLM-as-judge evaluation:** Replace or augment rule-based metrics with an LLM evaluator that checks factual plausibility, date consistency, and geographic feasibility which catches failures the current metrics miss.
- **Multi-turn dialogue:** The current system processes a single query. Adding conversation history would enable the agent to handle follow-ups like 'change the hotel to somewhere closer to the Eiffel Tower' without restarting from scratch.
- **Live mode evaluation at scale:** The 40-scenario evaluation used mock data for reproducibility. Running the same harness in live mode would test whether the API integration failures (fallback rate, latency variance) change the strategy rankings.
- **Expanded destination coverage:** Mock data currently covers only Paris, Tokyo, and Rome. Adding 10–15 cities from underrepresented regions would better test geographic generalization and surface potential bias in the recommendation layer.

6. References

[1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing reasoning and acting in language models*. International Conference on Learning Representations. https://arxiv.org/abs/2210.03629

[2] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., & Lim, E.-P. (2023). *Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models*. Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics. https://aclanthology.org/2023.acl-long.147/

[3] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). *Reflexion: Language agents with verbal reinforcement learning*. Advances in Neural Information Processing Systems, 36. https://arxiv.org/abs/2303.11366

[4] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). *Chain-of-thought prompting elicits reasoning in large language models*. Advances in Neural Information Processing Systems, 35. https://arxiv.org/abs/2201.11903

[5] OpenAI. (n.d.). *Function calling*. OpenAI API documentation. Retrieved May 1, 2026, from https://platform.openai.com/docs/guides/function-calling

[6] SerpAPI. (n.d.). *Google Flights API*. Retrieved May 1, 2026, from https://serpapi.com/google-flights-api

[7] OpenWeather. (n.d.). *5 day weather forecast*. Retrieved May 1, 2026, from https://openweathermap.org/forecast5

[8] Geoapify. (n.d.). *Places API documentation*. Retrieved May 1, 2026, from https://apidocs.geoapify.com/docs/places/

[9] DataCrawler. (n.d.). *Booking.com API*. RapidAPI. Retrieved May 1, 2026, from https://rapidapi.com/DataCrawler/api/booking-com15